

SESSION NUMBER – 12

TASK 1 — Producer-Consumer using Semaphores

Question

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Create folder

```
mkdir -p session12
cd session12
ganesh@DESKTOP-79N3772:~/session11$ mkdir -p session12
ganesh@DESKTOP-79N3772:~/session11$ cd session12
ganesh@DESKTOP-79N3772:~/session11/session12$
```

C program:

```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
    }
}
```

```
        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}
```

```
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output:

```
ganesh@DESKTOP-79N3772:~/session11/session12$ nano producer_consumer.c
ganesh@DESKTOP-79N3772:~/session11/session12$ gcc producer_consumer.c -o producer_consumer -pthread
ganesh@DESKTOP-79N3772:~/session11/session12$ ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
ganesh@DESKTOP-79N3772:~/session11/session12$
```

Observation

Producer and consumer threads were synchronized successfully using semaphores and mutex locks without data loss.

TASK 2 — Deadlock and Prevention using Resource Ordering

Question

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions, and implement a deadlock prevention strategy using resource ordering.

```
GNU nano 8.7.1                                deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource2);

    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

compile

```
consumed: 10
ganesh@DESKTOP-79N3772:~/session11/session12$ nano deadlock.c
ganesh@DESKTOP-79N3772:~/session11/session12$ gcc deadlock.c -o deadlock -pthread
ganesh@DESKTOP-79N3772:~/session11/session12$ ./deadlock
Thread 1 locked Resource 1
Thread 2 locked Resource 2
```

PART B — Deadlock Prevention using Resource Ordering

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    printf("Thread 1 completed\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");
```

```
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    printf("Thread 2 completed\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;
```

```
pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

Output:

```
ganesh@DESKTOP-79N3772:~/session11/session12$ nano no_deadlock.c
ganesh@DESKTOP-79N3772:~/session11/session12$ gcc no_deadlock.c -o no_deadlock -pthread
ganesh@DESKTOP-79N3772:~/session11/session12$ ./no_deadlock
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 1 completed
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 2 completed
ganesh@DESKTOP-79N3772:~/session11/session12$
```

Four Necessary Conditions for Deadlock

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Observation

Deadlock was demonstrated using two resources. Resource ordering prevented circular wait and successfully avoided deadlock.

